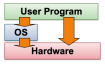
CS2106 Cheatsheet, AY26/27 S1

Zhao Wu and Tien Cheng

October 24, 2026

1 Introduction

OS is an **intermediary program** between **computer hardware** and **user**.

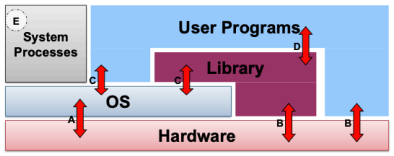
First Computer	**OS Type**: No OS • Program directly by physically changing the hardware configuration (cables, switches, punched paper tape) **Advantage**: • Minimal OS overhead **Disadvantage**: • Not portable (to port, need to manually rewrite the program) and Inefficient • E.g. Electronic Numerical Integrator And Computer (ENIAC), Harvard Mark I	
Mainframes	**OS Type**: Batch OS • Execute one job at a time. • Support batch processing only • No interactive interface (program via paper tape, magnetic tape, punch card) • User interacts with hardware directly with additional information for OS (e.g. resource required, job specification) **Disadvantage**: • CPU is idle during I/O from simple batch processing • E.g. IBM 360	
Time-Sharing OS	**OS Type**: Time-Sharing OS **Advantage**: • Allow multiple users to interact with the machine using terminals (teletypes) • User job scheduling (illusion of concurrency) • OS manages CPU time, memory and storage • Virtualization of hardware (each program executes as if it has all the resources to itself) • E.g. Apple II PC, IBM PC	
Personal OS	Machine may be dedicated to user, not timeshared between users Windows model: • Single user at a time, but possibly more than 1 user can access Unix model (General time-sharing model): • One user at the workstation, but other users can access remotely	

Motivation of OS

1. **Abstraction**: Hide low-level details and present higher-level functionality to the user
2. **Resource Allocator**: Manage all resources (CPU, Memory, I/O) and arbitrate potentially conflicting requests for efficient and fair resource use
3. **Control Program**: Control execution of programs to prevent errors and improper use of the computer

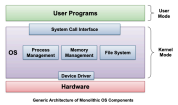
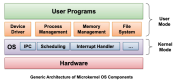
1.1 OS Structures

- OS is software that runs in **Kernel Mode** with direct access to all hardware resources (w/o system calls, normal libraries and 'normal' I/O in kernel mode)
- Other software operates in **User Mode** (i.e. limited/controlled access to hardware resources)



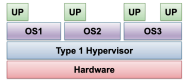
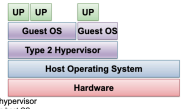
- **A**: OS executing machine instructions
- **B**: Normal machine instructions executed (program/library code)
- **C**: Calling OS using ***system call interface***
- **D**: User program calls library code
- **E**: System processes
  - Provide high-level services, usually part of OS

OS Structures

Monolithic	• Big program (e.g. Linux source code) • If Kernel fails, BSOD • Better performance	
Microkernel	• Smaller and cleaner abstraction • Provides basic and essential facilities: ‣ Inter-Process Communication ‣ Address space management ‣ Thread management • Higher-level OS services are run outside of the kernel, using IPC to communicate. • Lower performance	

1.2 Virtual Machines

A **Virtual Machine** emulates the underlying hardware for running multiple OS, created and managed by a **Hypervisor** / **Virtual Machine Monitor (VMM)**

Type 1 Hypervisor	Type 2 Hypervisor
	
▪ Type 1 hypervisor: ▫ AKA bare-metal hypervisor ▫ Provides individual *virtual machines* to guest OSes ▫ E.g., IBM VM/370, VMware ESXi	▪ Type 2 hypervisor ▫ Runs in host OS ▫ Guest OS runs inside Virtual Machine ▫ e.g., VMware Workstation, VirtualBox, QEMU

2 Process Abstraction

- **Process** is the OS abstraction for a running program.

Process Context

Layer	Component	Stores
Memory	Text	program instructions
	Data	global variables and static variables
	Stack	collection of stack frames
	Heap	region of memory used to store dynamically allocated data
Hardware	General-Purpose Registers (GPRs)	temporary data
	Program Counter (PC)	next instruction address
	Stack Pointer (SP)	top of stack frame address
	Frame Pointer (FP)	fixed location in stack frame
OS	Process ID (PID)	unique process ID
	Process State	current process state

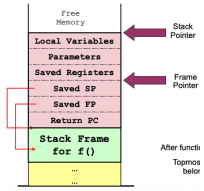
2.1 Memory & Function Calls

- Each function invocation **pushes** / return **pop** a new **stack frame onto the stack**.
- **FP** points to the fixed location in stack frame, allowing access with constant offset.

Stack-Frame Component	Who	Why
Local variables	Callee	Callee's own local variables
Parameters (only those that don't fit in registers)	**Caller**	so the callee can read the parameters of function call
Saved GPRs	Callee	Copy of GPRs which callee modifies; protect the caller's values across the call
Saved old SP	Callee	restore the caller's SP
Saved old FP	Callee	restore the caller's FP
Return address / Saved PC	**Caller**	return to the right instruction after the callee returns

Function Call Convention

1. **Function Call Preparation**:
   1. Caller: pass **parameters using registers and/or the stack**
   2. Caller: save **return PC on stack**
2. **Transfer of Control from Caller to Callee**:
   1. Callee: save **registers used by the callee, old SP and FP**
   2. Callee: allocate space for **local variables of the callee on the stack**
   3. Callee: adjust the **SP (and FP) to point to the new stack top**
3. **Callee Function Executes**
4. **Callee Function Returns**
   1. Callee: save **return result in return register** (if applicable)
   2. Callee: **restore saved registers and SP, FP**
5. **Transfer of Control from Callee back to Caller using saved PC**:
   1. Caller: Use return result and continues execution of the program



1. Return Value

Register Saving and Spilling

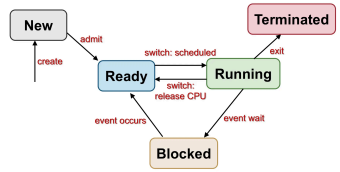
- **Register Saving**: caller and callee may use the same registers, so we need to save the registers and restore them after the call.
- **Register Spilling**: When a function has more arguments than the number of registers, the extra arguments are spilled to the stack
  - **Callee-saved** (default 2106 convention): the callee saves the old values of the registers into its stack frame and restores them after the call
  - **Caller-saved**: the caller saves needed registers into its stack frame and restores them after the call
- **Stack Frame Deallocation**: doesnt guarantee removing of previous values

Heap Memory

- **heap** is a memory region to store dynamically allocated memory. (e.g. `malloc` and `free` in C)
- Dynamically allocated data **cannot be stored in**:
  - **Data region**: size must be known at compile time
  - **Stack**: the data may outlast the function call

2.2 Process States

Process State Model



- Voluntarily give up cpu: running → ready
- Context switch: ready → running

Multi-Process Management

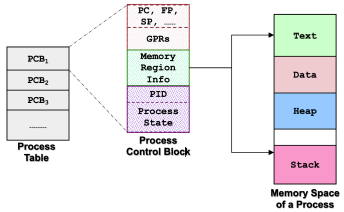
- With 1 CPU core, at most 1 process can be running at a time (without threading)
- With m CPU cores, at most m processes can be running at a time

Process Queues

- The OS maintains a queue of processes for each state
  - **Ready Queue**: processes that are ready to be scheduled to run on the CPU
  - **Blocked Queue**: processes that are waiting for an event to occur
    - may be separate queues for diff types of blocked (e.g. I/O / signal blocked)

2.3 Process Control Block (PCB)

A **Process Control Block (PCB)** or **Process Table Entry** is a data structure that **describes the execution context** for a **process**, maintained by kernel.



2.4 System Calls

System Call is a synchronous **OS API** for a **user program to request services** in Kernel.

System Call Mechanism

1. User invokes the library call (e.g. `getpid()`)
2. Library call places the system call number into a register.
3. Library call invokes the **trap instruction to switch from user mode to kernel mode**.
4. In kernel mode, the **dispatcher identifies the system call** and **passes control to the appropriate system call handler**.
5. System call handler executes the system call
6. System call handler ended by restoring CPU state and return to user mode.
7. Library call returns the result to the user program.

Exception and Interrupt

- **Exception**: Occurs due to program execution, synchronous
  - E.g. divide by zero error or a page fault
- **Interrupt**: Occurs independent of program execution, asynchronous
  - E.g. hardware interrupt or a software interrupt

2.5 A Case Study of Processes in Unix

2.5.1 Process Creation: Fork -> Exec

fork()

`int fork()` creates a child process by duplicating currently executing parent.

- Both parent and child continue from **immediately after** the `fork()` call.
- Child is initially an **almost exact duplicate**:
  - Same code and **initial** address-space **contents** (conceptually duplicated, **not** shared)
  - Copied register and execution context
  - Different PID and PPID
  - Different `fork()` return value: parent receives the child's PID; child receives `0`

Nondeterministic scheduling

- Parent/child order is **nondeterministic**: parent first, child first, or interleaved.

Independent Address Spaces

Parent and child initially **see equivalent values** after `fork()`, but do **not** share memory.

- Stack, heap, data, and code image are **conceptually duplicated**.
- Modifying a variable in one process does **not** modify the other.
- Shared kernel resources are inherited (e.g. opened files), not the address space.

exec() replaces the process image

`exec()` **replaces** the current process (code and data) with a new executable:

- Begins at the new program's entry point
- Discards the old stack and execution state
- **Retains the same PID** and broader process identity
- **Successful `exec()` does not return** — the old image no longer exists
- Variants: `execl(full path, arg0, ..., NULL)`, `execv("/bin/ls", args)`, `execve`, `execlp(relative path, arg0, ..., NULL)`, `execvp`

main Command-Line Arguments

`int main(int argc, char **argv[], char **vp)`

- `argc`: number of arguments, **including** the program name
- `argv`: array of C strings arguments; `argv[0]` is conventionally the executable name
- `vp`: array of C strings environment variables

init and Process Tree

A process can only be created by forking an existing process, so Unix processes form a **process tree**.

- Root is **init**: created by the kernel during boot, traditionally **PID 1**
- Common ancestor of user processes; typically spawns OS/system programs
- Adopts orphaned processes
- **Cannot be killed** even though it runs in user space; if it crashes → **kernel panic**

2.5.2 Process Termination & Lifecycle

exit()

`exit(int status)` terminates the current process. **Does not return**.

- **exit status**: zero if normal/successful termination, non-zero if error/abnormal
- Returning from `main()` **implicitly** invokes `exit()` with return value becomes the exit status
- On process `exit()`:
  - Process state is set to **Zombie**
  - Most resources are released (e.g. file descriptors)
  - Some information is **not releasable**, so the parent can `wait()`, Eg: PID, Exit status, Process accounting (e.g. CPU time)

wait()

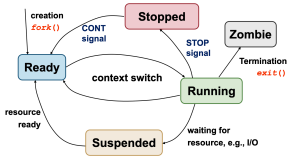
`pid_t wait(int *status)` lets a parent synchronise with any child(s):

- **Blocks** until at least one child terminates
- Returns the PID of a terminated child
- Stores the child's exit status through `status`, else `NULL`
- Kernel can write into the parent's memory because it is privileged
- Variants:
  - `waitpid()` — wait for a specific child
  - `waitid()` — wait for child state changes

Zombie vs Orphan

- **Zombie**: child that has **exited** but **has not yet been consumed** by parent via `wait()`
  - Occupies a process-table entry; too many can exhaust the table (older Unix: may need reboot)
- **Orphan**: a **still-running** child whose **parent has terminated**
  - `init` becomes its pseudo-parent
  - When the child later terminates, `init` will clean up with `wait()`
  - If the parent dies while the child is **already a zombie**, `init` reaps that leftover state too

Unix Process States



3 Process Scheduling

Concurrent processes execute during the same period:

- **Virtual parallelism:** pseudo-parallelism.
- **Physical parallelism:** multiple CPUs/cores execute processes in parallel.

Time slicing interleaves processes' instructions and requires a **context switch** at each handover. It works on **single core** (slice instructions) or **multiple processors** (slice instructions and cores).

Scheduling

The **scheduler** uses a **scheduling algorithm** to choose the next process:

- **Non-preemptive (cooperative):** runs until it blocks or voluntarily yields the CPU.
- **Preemptive:** runs for a fixed quota, then is suspended.

3.1 Scheduling for Batch Processing

Batch systems have **no user interaction** and mainly use **non-preemptive scheduling**.

Criteria:

- **turnaround time** = finish - arrival
- **throughput** = tasks finished per unit time
- **waiting time** = total time waiting since creation
- **CPU utilization** = percentage of time the CPU runs a task.

First Come First Serve (FCFS)

Uses a **FIFO queue**. Its head runs until **completion or blocking**; a blocked task leaves the queue and rejoins its **tail** when ready.

- **Traits: non-preemptive, no starvation.**
- **Costs:** the **convoy effect** prevents CPU- and I/O-bound processes from running concurrently; **reordering** tasks can lower average waiting time.

Shortest Job First (SJF)

Non-preemptively selects the task with the **smallest total CPU time**.

- Requires **advance knowledge** of each task's total CPU time.
- **Minimizes average waiting time**, but its bias toward short jobs can **starve long jobs**.

Predicting CPU Time

Estimate the next CPU-bound phase from previous phases:

$$\text{Predicted}_{n+1} = \alpha\text{Actual}_n + (1-\alpha)\text{Predicted}_n$$

Actual_n: most recent CPU time; Predicted_n: past estimate; α: weight on the recent event; Predicted_{n+1}: new estimate.

Shortest Remaining Time (SRT)

Preemptively runs the job with the **shortest remaining (or expected) time**; a newly arrived shorter job **preempts** the running job.

3.2 Scheduling for Interactive Environment

Interactive systems mainly use a **periodically invoked, preemptive scheduler** for good **response time** and **predictability**.

Response Time: First CPU time - time task creation

Timer & Time Quantum

- **Timer interrupt interval (ITI):** interval between interrupts that invoke the OS scheduler; typically **1–10 ms**.
- **Time quantum:** process execution allowance; a **multiple of the ITI**, constant or process-dependent, typically **5–100 ms**.

Round Robin (RR)

Preemptive FCFS using a **FIFO queue**. Its head runs until its **quantum expires, it finishes, or it blocks**; an unfinished task rejoins the tail, as does a blocked task once ready.

- **Timer interrupts** detect quantum expiry.
- **Wait bound:** $(n-1)q$ before getting the CPU, for n tasks and quantum q.
- **Large q:** higher CPU utilization, longer waits. **Small q:** more overhead, shorter waits.

Priority Based

Runs the task with the **highest priority value**.

- **Preemptive:** a higher-priority arrival preempts a lower-priority running process.
- **Non-preemptive:** a late higher-priority arrival waits for the next scheduling round.
- **Starvation:** low-priority processes may never run. Mitigate by **lowering the running process's priority** after each quantum, or **excluding it from the next round** after giving it a quantum.
- **Priority Inversion:** H priority task is blocked by L priority task, who will never get scheduled as M priority task is in the way.

Multi-Level Feedback Queue (MLFQ)

Higher priority wins; equal priorities use RR. New job -> highest priority, Deplete time quantum -> priority decrease, Job gives up -> priority maintained. This minimizes **response time for I/O-bound** processes and **turnaround time for CPU-bound** processes.

- Constantly spawning child before time quantum exceeds
- Bias for IO heavy task, CPU heavy task may starve

Lottery Scheduling

Distribute **lottery tickets** for resources (CPU time, I/O devices). At each decision, **randomly choose** an eligible ticket; in the long run, **X% of tickets gives X% resource use**.

4 Inter-Process Communication

Inter-Process Communication transfers information between processes.

Shared-Memory

A **shared memory** region M is created(`shmget`) and attached(`shmat`) by the processes P_1 and P_2, enabling communication.

- Efficient: OS only needs to Create and Attach memory region
- `shm[0]` to declare if the value is ready
- `shmget` to create, `shmat` to attach, `shmdt` to detach and `shmctl` to destroy

Message Passing

Process P_1 prepare messages M(stored in **kernel memory space**) and send to P_2 using **system calls**. Requires extra properties like Name and Synchronization.

- **Direct Communication**: Sender and Receiver 1-1 message explicitly name the other party (ie. unix domain socket). `Send/Receive( P1, Msg )`
- **Indirect Communication**: Messages are sent / received from many-many message storage known as mailbox or port (i.e. unix message queue). `Send/Receive( MB, Msg )`
- **Blocking Primitives (synchronous)**: Receiver is blocked until message has arrived.
- **Non-Blocking Primitives (asynchronous)**: Receiver either receive the message if available or some indication that message is not ready.

4.1 Unix Pipes and Signals

A process has 3 default communication channels: stdin (`scanf`), stdout (`printf`), stderr. Declared in bash with "|".

`int pipe(int fd[])` functions as circular bounded byte buffer (**writers wait when buffer is full**) and implicit synchronization (**readers wait when buffer is empty**)

- Returns −1 if creation of fd fails
- Returns EOF if read from an empty pipe with no writer

Steps for creating a pipe:

- `pipe(fd[2])` creates an array of file descriptor with `READ_END=0`, `WRITE_END=1`.
- process may `write(fd[WRITE_END], str, strlen(str)+1)` or `read(df[READ_END], buf, sizeof(buf))`
- `dup2(fd[0], STDID_FILENO)` and `dup2(fd[1], STDOUT_FILENO)`

Redirecting `printf` to `STDOUT`:

1. `close(STDOUT);`
2. `open("file.txt", O_WRONLY | OCREATE | OTRUNC, 0644);`

Unix Signal

`signal` sent to a process using an **asynchronous** notification regarding an event.

- Eg. Kill, Interrupt, Stop, Continue, Memory Error, Arithmetic Error…
- `kill -9` (ie SIGKILL) cannot be captured with user-define handler
- Signal handler cannot wait or signal a semaphore as it is not async safe

4.2 Threads

Multiple **threads** run concurrently in a process, sharing the **same memory context** (e.g. text, data, heap) and **OS context** (PID and other resources), **differing** only in **hardware context** (registers and stack).

User and Kernel Thread

- **User Thread** is thread implemented as user library and handles by its runtime process, not aware by kernel.
  - Pros: More configurable and portable as thread operation are just library calls.
  - Cons: Scheduling performed at process level (1 thread blocks out whole process); cannot exploit multiple CPU.
- **Kernel Thread** is thread implemented by OS, handled as system calls, enabling thread-level scheduling.
  - Pros: More than 1 thread in same process can run on multiple CPUs as it is handled by Kernel
  - Cons: Thread operation with system call (higher overhead), and generally less flexible.
- **Hybrid Thread** uses both user thread and kernel thread at the same time.

5 Synchronization

Concurrent execution is **non-deterministic**, so a **race condition** occurs when the outcome depends on the **order of shared-resource access/modification**.

Critical section (CS) is code segment that only one process may execute at a time with the following properties

- **Mutual Exclusion**: Only one processes can executes CS.
- **Progress**: If no process in CS, one of the waiting processes is granted access.
- **Bounded Wait**: Upperbound time for P_1 in CS, before other process can enter.
- **Independence**: Process in non-CS should never block other process.

Outcome of incorrect CS implementation:

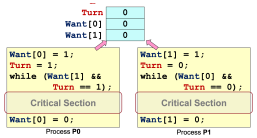
- **Deadlock**: All processes blocked and stops exeecution
- **Livelock**: Result of deadlock avoidance mechanism, where processes keep changing state and make no progress
- **Starvation**: Some processes are blocked forever

Low-Level Implementation: TestAndSet

`TestAndSet Register, MemoryLocation` loads the content from memory to register, and stores 1 to memory, happens on hardware level (atomic).

```
EnterCS(int* Lock) while (TestAndSet(Lock) == 1) // Loop -> unlock
ExitCS(int* Lock) *Lock=0; // Set lock to 0
```

Higher level Language: Peterson's Algorithm



Higher level synchronization: Semaphore

Sempahore is an integer $S \geq 0$, represent number of jobs that can run concurrently (counting semaphore). `mutex` when $S \in \{0,1\}$

- `Wait(S)`: When S==0, blocks and puts process to sleeping queue. Else, S– (`P()`, `Down()`)
- `Signal(S)`: Wakes up 1 sleeping processes if any. Else, S++ (`V()`, `Up()`)

$$N_{CS} = \#\text{Signal(S)} - \#\text{Wait(S)} \quad S_{\text{current}} = 1 + N_{CS}, \quad N_{CS} \leq S_{\text{initial}}$$

6 Memory Management

Random Access Memory is physical memory storage, consisting of array of bytes, identifiable by its physical address.

- **Transient Data**: Valid only for limited duration, e.g. function call.
- **Persistent Data**: Valid for the duration of program unless explicitly removed.

6.1 Memory Abstraction

Without **memory abstraction**, programs access **physical memory directly**, so **multiple processes of the same program** may conflict.

Following are few attempts for memory abstractions and their tradeoffs:

1. **Address Relocation**: Adds **fixed offset** to all memory reference **when process is loaded to memory**.
   - Slow loading time and inability to distinguish memory reference from integer constant
2. **Base + Limit Registers** uses `base` and `limit` registers to offset and set limit for memory access and boundary on memory access.
   - `Actual = Base + Adr && Actual < Limit`
   - Every memory access incur additional + and < operation
3. **Logical Address** is memory space accessible by a process.
   - Requires mapping between logical address and physical address
   - Self-contained, independent logical memory space for each process

6.2 Contiguous Memory Allocation

Memory Partition is continuguous memory region (i.e. interval of consecutive addresses) allocated to a single process.

Fixed Partitioning

Fixed-Size Partition splits physical memory into fixed number of partitions.

- Easy to manage / allocate
- Partition size need to be large enough to contain the largest processes, resulting in **internal fragmentation** (gap within partition)

Dynamic Partitioning

Dynamic Partitioning allocate memory space to free memory space (aka hole) based on allocation algorithm.

Algorithms to locate partition of size N:

1. First-Fit: take the **first hole** that is large enough
2. Best-Fit: take the **smallest hole** that is large enough
3. Worst-Fit: take the **largest hole**

Compaction may be used to move occupied partition around to create consolidated holes.

Buddy System

Buddy memory allocation splits hole into half repeatedly to meet the best-fit request, and merge the buddies together when both are free.

- provides efficient partition splitting
- locating good match of hole
- partition de-allocation and coalescing.
- may result in **external framentation** (gap between partition)

Initialisation:

1. Maintain an array A of size K where each i-th element consists of linkedList of free blocks of size 2^i

Allocation:

1. Given a block request of size N, compute $S = \lceil \log_2 N \rceil$ which is the minimum size.
2. If `A[S]` is not empty, allocate the block by remove the first block from the linkedList.
3. Else:
   1. Find smallest $R \in [S+1, K]$ where `A[R]` is not empty
   2. For $j \in [R-1, S]$ (reverse order), repeatedly split B to create free block

Deallocation:

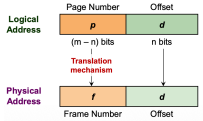
1. Append free block to `A[S]`
2. If the buddy block exists in `A[S]` (identified where all bits are the same except for bit), merge them and go to Step 1

6.3 Disjoint Memory Allocation

Paging is process of mapping **logical pages** $\in$ **logical memory** to **physical frames** $\in$ **physical memory**, in disjointed physical memory locations.

$$\text{Physical Frame} = \text{PageTable}[\text{LogicalPage}] \times 2^{n} \text{ (Frame Size)} + \text{Offset}$$

- Page Table: Maps page number $p = m - n$ to frame number f. Size of 2^{m-n}
- Frame size: Size of physical frame and logical page, Size of 2^n.



Key characteristics:

- Page table is **stored in PCB**, but requires **two memory access** for every memory reference (one for page table entry, another for actual memory item).
  - Access Right Bits: indicate if data is writable, readable and executable
  - Valid Bit: Indicate the page is valid for process to access
- **Removes external fragmentation**, but may **have internal fragmentation**.

Translation Look-Aside Buffer (TLB)

Translation Look-Aside Buffer (TLB) is **hardware context** of process (flushed when context switch occurs) that **caches a few page table entries**.

- `TLB-Hit`: Frame number is retrieved to generate physical address.
- `TLB-Miss`: Memory access to access the full page table, update TLB after retrieving frame number.

Page Sharing

Page Sharing allows proccesses to share the same physical memory frame in page table, enabling:

- Shared code page: system code that are used by many processes
- Copy-on-write: to allow child processes to share a page until the values is mutated

Segmentation

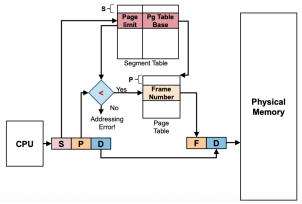
Segmentation seperates memory of a process into different memory segments (e.g. *text, data, heap, stack*) $\in$ logical memory space, with each segments identifiable with a segment name and limit.

Segment table, identifiable by sgement id, stores the base address and limit of a segment.

$$\text{Physical Address} = (\text{SegmentTable}[\text{SegId}].\text{Base} + \text{Offset}) \text{ Offset} < \text{Limit}$$

Segmentation with Paging

Store each memory segment in disjointed logical memory pages



6.4 Virtual Memory Management

Virtual memory saves to page table entries in secondary storage (e.g harddisk), while the rest are in physical memory.

Memory resident refers to page table entry that exists in physical memory, while **non-memory resident** are in secondary storage, `isMemoryResident` stores in page table.

Page Fault when CPU access non-memory resident page (before bringing page into physical memory, aka **Trashing**).

Locality Principles:

- Temporal Locality: memory address used is likely to be used again.
- Spatial Locality: memory address close to a used address is likely to be used.

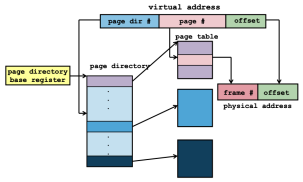
Page Table Structure

Direct Paging keeps all page table entries in a single table, with size of 2^p

- However, this resulted in the whole page table being stored in physical memory, when we dont need that many page table entries active at one time.

2-Level Paging extends direct paging, by using first m bit for outer page table/page directory, and $p - m$ bit for inner/smaller page table.

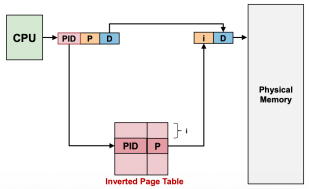
- Empty entries in page directory is possible
- Only page directory needs to be in memory



Inverted Page Table uses process id and page number, to uniquely identify a frame number.

Since page number is process specific, each process loading its own page table is waste of space.

With process id, we can perform some hashing to obtain the physical frame number.



Page Replacement Algorithms

Page replacement happens when page fault occurs and physical memory frame is full.

$$T_{\text{access}} = (1-p) * T_{\text{mem}} + p * T_{\text{page_fault}}$$

- p: probability of page fault
- T_{mem}: access time for memory resident page
- $T_{\text{page_fault}}$: access time when page fault
- T_{access}: total memory access time

Optimal Page Replacement (OPT) replaces page that will not be used again for longest period of time.

- guarantees minimum page fault, but rely on future knowledge

FIFO uses a queue and replace oldest memory page.

- does not exploit temporal locality
- Belady's Anomaly: higher number of frames resulting in more page faults

Least Recently Used (LRU) replace page that has not been used in longest time

- exploit temporal locality
- does not suffer from Belady's Anomaly
- hard to implement using pure counter and stack

Second-Chance Page Replacement (CLOCK) modified a FIFO queue to give second chance to page that are accessed.

When a page is accessed, reference bit flips to 1. Oldest page has reference bit of 0 => evicted; reference bit of 1 => reference bit to 0

Frame Allocation

Given the full physical memory, how much frame should we allocate for each process affects the trashing.

Local Replacement: Fixed frame given for process, local trashing occur.

- Pro: Stable performance between runs
- Cons: Slow progress if frame is not allocated enough

Global Replacement: Self-adjustment of frame between processes

- Pro: Process that needs more frame can get from other
- Cons: Badly behave process can affect others and inconsistent frame allocation from run to run

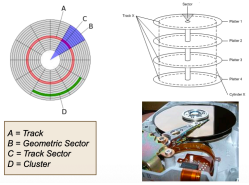
Working Set Model $W(t, \Delta)$ uses a window Δ, representing the amount of frame allocated to a process at given time interval t

E.g. when transiental period, increase Δ, when persistal period, lower Δ

7 File System

File system consist of collection of files and directory structures.

- provide abstraction for physical media
- high level resource management scheme, enabling
- Protection between processes and users
- Sharing between processes and users



	Memory Management	File System Management
Underlying Storage	RAM	Disk
Access Speed	Constant	Variable disk I/O time
Unit of Addressing	Physical memory address	Disk sector
Usage	Address space for process **Implicit** when process runs	Non-volatile data **Explicit** access
Organization	**Paging/Segmentation:** determined by HW & OS	Many different FS: ext* (Linux), FAT* (Windows), HFS* (Mac OS)etc.

7.1 File

File consist of Data and Metadata/File Attributes.

File Metadata

Name:	A human readable reference to the file
Identifier:	A unique id for the file used internally by FS
Type:	Indicate different type of files E.g. executable, text file, object file, directory etc
Size:	Current size of file (in bytes, words or blocks)
Protection:	Access permissions, can be classified as reading, writing and execution rights
Time, date and owner information:	Creation, last modification time, owner id etc
Table of content:	Information for the FS to determine how to access the file

1. **File Name**
   - Naming rule for different FS, including file name length, case sensitivity, special symbol, file extension postfix
2. **File Type**
   - File type has associated sets of file operations.
   - Distinguishable using file extension in Windows and Magic Number in Unix.
   - Common file types:
     - Regular Files: User data (ASCII files, Binary files)
     - Directories: System files for FS structure
     - Special Files: character/block oriented
3. **File Protection**
   - Control access to information stored in a file.
   - Type of Access: Read, Write, Execute, Append, Delete, List
   - Permission Bits: Owner(rwx), Group(r- -), Universe (r- -)
   - Access Control List:

```
getfacl exampleDir
# file: exampleDir
# owner: sooyj
# group: compsc
user::rwx
user:aaron:rwx
group::r-x
group:cohort17:rwx
mask::rwx
other::---
```

File Data

Each file is array of bytes, with offset from the start of file.

File Access Methods:

1. Sequential Access: read data in order, starting from beginning
2. Random Access:
   1. Read (Offset): Read operation from position stated
   2. Seek (Offset): Move to a new location in file
3. Direct Access: Allow random access to any record directly, for file with fixed-length records. Useful for large amount of records.

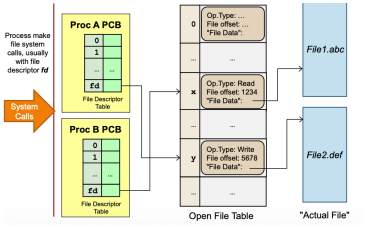
File operation

OS provides system calls for file operation.

Create:	New file is created with no data
Open:	Performed before further operations To prepare the necessary information for file operations later
Read:	Read data from file, usually starting from current position
Write:	Write data to file, usually starting from current position
Repositioning:	Also known as seek Move the current position to a new location No actual Read/Write is performed
Truncate:	Removes data between specified position to end of file

File table storing additional information for an opened file, identifiable by file descriptor, stored in PCB of each process.

- File Pointer: current location in file
- Disk Location: Actual file location on disk
- Open Count: number of processes opening the file



7.2 Directory

Directory provides logical grouping of files.

1. Single-Level: Every file exists in a single directory
2. Tree-Structure: Directories can recursively embedded in other directories
3. DAG: Tree-Structure with special hard link (`ln`, file only) / symbolic link (`ln -s` file or directory) from directory
4. General Graph: DAG with cycles. Not desirable but can be created with symbolic link in unix.

7.3 Unix File Operation

Uses `<sys/types.h>`, `<sys/stat.h>`, `<fcntl.h>`. Files are accessed **byte-by-byte** (no data interpretation), identified by a **file descriptor** (integer).

open()

`int open(char *path, int flags)` opens/creates `path`, returns a **file descriptor** (≥ 0) or -1 on failure.

- `flags`: bitwise-OR'd options — read/write/read+write mode, truncate, append, create if not exists, etc.
- e.g. `open("data.txt", O_RDONLY)`, `open("data.txt", O_RDWR | O_CREAT)`
- Default fds by convention: `STDIN` (0), `STDOUT` (1), `STDERR` (2)

read()

`int read(int fd, void *buf, int n)` reads up to `n` bytes from the **current offset** into `buf`. Returns bytes read ($0..n$), where $< n$ means **EOF reached**.

- **Sequential**: starts at current offset, increments it by bytes read

write()

`int write(int fd, void *buf, int n)` writes up to `n` bytes from `buf` at the **current offset**. Returns bytes written (≥ 0), or -1 on error (e.g. exceeds file size limit, quota, disk space).

- **Sequential**: starts at current offset, increments it by bytes written; can grow the file past EOF to **append** data

lseek()

`off_t lseek(int fd, off_t offset, int whence)` moves the current position by `offset`. Returns new offset, or -1 on error. Can seek **beyond EOF**.

- `offset`: positive = forward, negative = backward
- `whence`: `SEEK_SET` (absolute, from start), `SEEK_CUR` (relative to current), `SEEK_END` (relative to end)

close()

`int close(int fd)` closes `fd`: it becomes **reusable**, kernel **frees associated data structures**. Returns 0 on success, -1 on error.

- Process termination **automatically closes** all open files